ResQ Helmet:
Crash Detection & Emergency Alert System For Riders

Embedded System and Interfacing
EMJ27204

LECTURERS: Assoc. Prof. Dr. Norasmadi Abdul Rahim
Dr. Ahmad Shakaff Ali Yeon

Student's Particular:

Name	Matric No.
1) MUHAMMAD IQBAL BIN IZAN	241061475
2) MUHAMMAD LUQMAN HAKIM BIN ABDUL RAZAK	241061477
3)MOHAMAD NAJMUDIN HAKIMI BIN MOHD MOHAIDIN	241061443
4) HAIRIE AQMAL BIN DANNIE AZLAN	241061434

SDG:11

YouTube Link:

1. Introduction

The integration of embedded systems into personal safety gear is revolutionizing how we protect lives on the road. With motorcyclists being one of the most vulnerable groups in road traffic accidents, there is a growing demand for real-time accident detection and faster emergency response systems. In most severe crash cases, victims are unable to alert emergency services themselves due to unconsciousness, leading to delayed medical assistance.

The ResQ Helmet is a smart helmet system designed to automatically detect motorcycle accidents and send emergency SOS alerts, along with real-time GPS location, to a pre-defined contact.

Key components such as MPU6050 (accelerometer + gyroscope) for crash detection and GPS for location tracking and messaging, and a microcontroller (Pico W) to manage system logic are integrated into a compact, wearable unit. This project is aligned with Sustainable Development Goal 11 (SDG 11): Sustainable Cities and Communities, which includes improving road safety through innovative technologies.

Ultimately, ResQ Helmet aims to minimize fatalities by ensuring faster emergency response and enhancing rider safety using reliable embedded technology.

2. Objectives

The objective of this project is to design a smart helmet system 'ResQ Helmet' that enhances motorcyclist safety through automated crash detection and emergency alert features

i. To develop a smart safety helmet that activates only when worn securely using a buckle detection system (limit switch).
ii. To implement crash detection using the MPU6050 sensor by monitoring acceleration and gyroscope thresholds.
iii. To provide real-time emergency alerts by sending the rider's GPS location via Telegram in the event of a crash.
iv. To enhance night-time visibility using an LDR-controlled red LED that automatically turns on in low-light conditions.

The overall goal is to provide an affordable and effective solution that supports quick emergency response and promotes road safety

3. Scope

- Focus on motorcyclist safety enhancement.
- Use of wearable embedded system.
- Real-time accident detection and emergency SMS alert.
- Activation only when helmet is worn
- Low-cost implementation using accessible hardware.
- Automatic night-time safety light using LDR.

4. Problem Statement

Many motorcyclists involved in accidents are unconscious and unable to contact emergency services. This delay in response increases the risk of fatality. The ResQ Helmet addresses this by:

i. Many motorcyclists do not wear helmets properly, reducing the effectiveness of safety systems.
ii. Delayed emergency response after accidents can lead to severe consequences or loss of life.
iii. Traditional helmets lack crash detection and location tracking features.
iv. Riders often remain unnoticed after a crash, especially in remote or dark areas.
v. SDG 11 aims to promote safety and sustainability in urban transportation systems.

5. Literature Review

Recent advances in embedded systems and IoT have made it possible to build smart safety devices like the ResQ Helmet. Many researchers have combined sensors, GPS, and wireless communication to improve accident detection and emergency response.

One study by Rajalakshmi and Padmavathi (2019) created a smart helmet that sends SMS alerts with location after a crash. The ResQ Helmet improves this by using a Telegram bot, which sends faster internet-based alerts.

Yogeswari et al. (2022) used an LDR sensor to control street lights at night. This inspired our use of a night LED that turns on in low light to make the rider more visible.

To avoid false alerts, the ResQ Helmet includes a limit switch, similar to Sunil et al. (2020), who used physical switches in industrial gear.

Lastly, the ResQ Helmet uses an RTOS (Real-Time Operating System) to manage multiple sensors efficiently. This idea is supported by Ghosh and Kumar (2020), who showed how RTOS helps wearable devices run tasks in real time.

Overall, these studies support the ResQ Helmet's design and show how smart technology can help keep motorcyclists safer.

6. Methodology

6.1 SYSTEM COMPONENTS

The ResQ Helmet system is built around two Raspberry Pi Pico W microcontrollers. One microcontroller is responsible for handling all sensor inputs and output signals such as the buzzer and LEDs. The second microcontroller focuses on communication modules, specifically the GPS components, ensuring smooth and uninterrupted data transmission..

6.2 SENSORS

The system includes an MPU6050 accelerometer and gyroscope to detect any sudden impact or abnormal helmet movement. A limit switch is used to determine whether the helmet is being buckle the system only activates when the switch is pressed. Additionally, a Light Dependent Resistor (LDR) monitors the surrounding light conditions and activates a red LED for night visibility when low light is.

6.3 CONTROLLERS

Raspberry Pi Pico W units are used to divide processing tasks efficiently. The first Pico W handles sensor readings, switch logic, LDR input, and controls the red alert LED and buzzer. The second Pico W operates supply power the GPS module , ensuring fast and reliable transmission of location data via telegram..

6.4 ACTUATORS

The buzzer and yellow LED are activated upon crash detection to serve as immediate alerts. A red LED is triggered by the LDR at night to improve visibility and alert nearby drivers. The system also includes a cancel button to stop alerts if no real accident occurred, reducing the risk of false notification**s.**

6.5 IOT PLATFORM INTEGRATION

Instead of relying solely on SMS, the system also uses Telegram as an IoT messaging platform. The GPS data is relayed through Telegram messages, allowing emergency contacts to receive crash notifications along with real-time location data in a more accessible and traceable format

6.6 OPERATION LOGIC

When the helmet is worn, the limit switch activates the system. The LDR continuously checks for low light levels and activates the night LED if needed. The MPU6050 monitors for crash-like movements. If a crash is detected, the buzzer and yellow LED are triggered. If the cancel button is not pressed within a few seconds, the GPS module captures the coordinates and the Telegram system sends the location to an emergency contact.This structured methodology ensures that all system functions operate independently yet synchronously, providing real-time safety responses tailored to real-world accident scenarios.

.

6.7 SCHEMATIC DIAGRAM

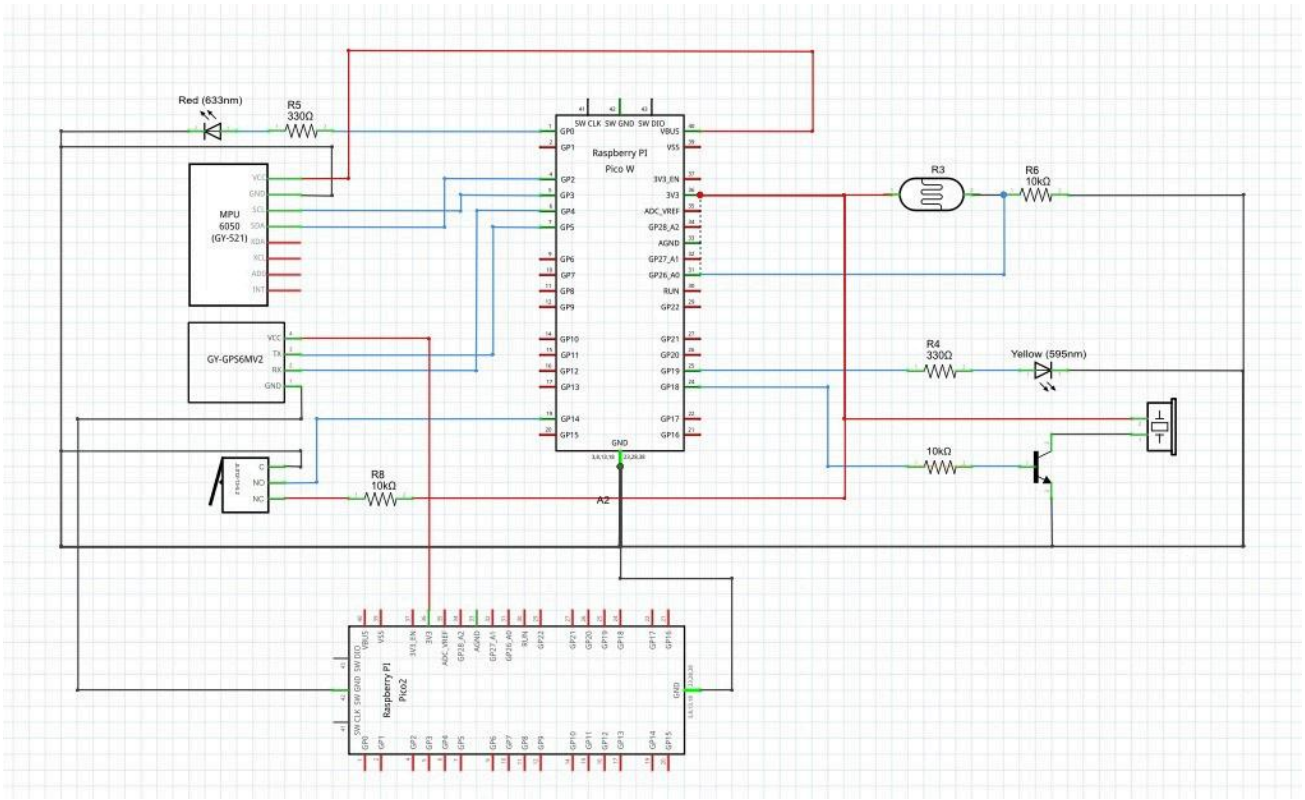

Diagram 1.1 Schematic

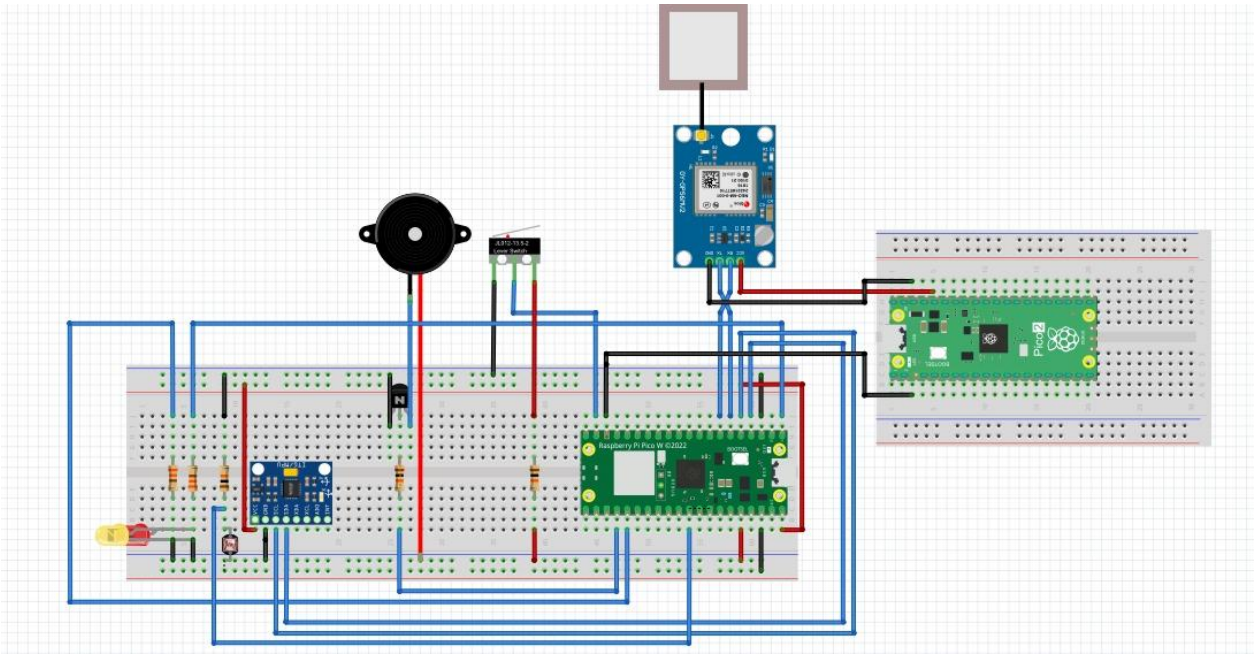

Diagram 1.2 Schematic Board

6.8 FLOWCHART

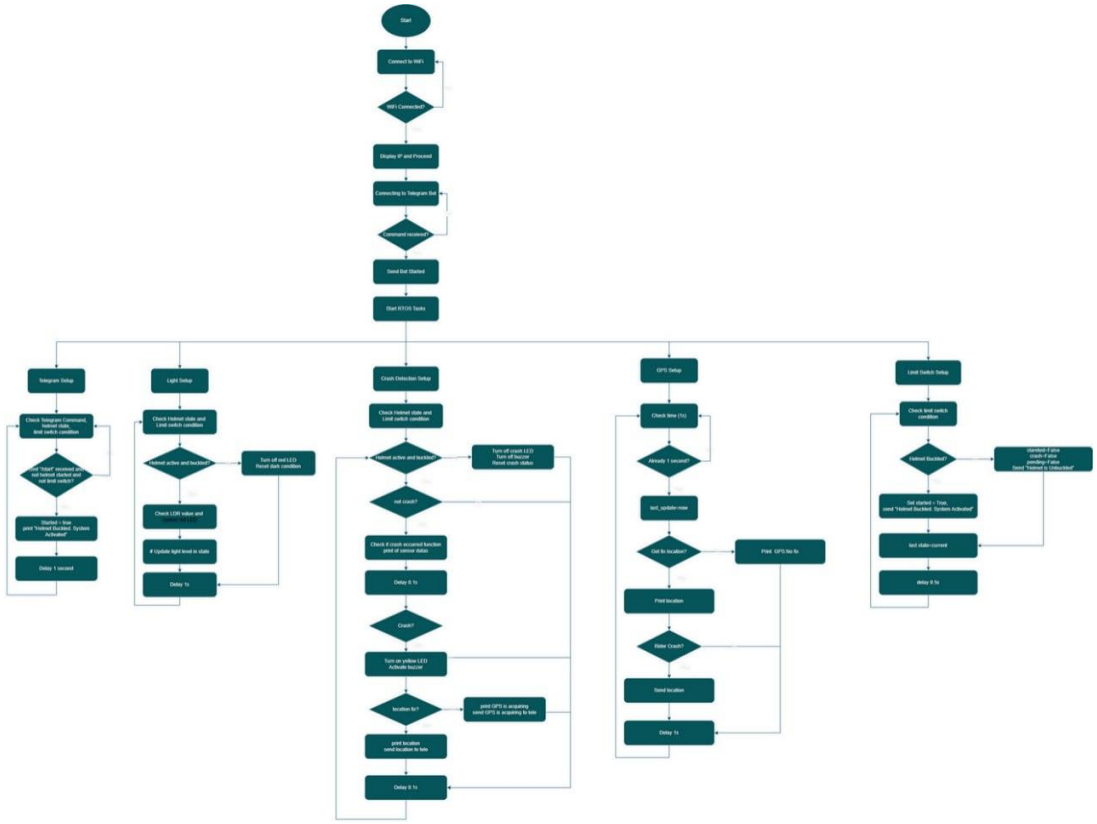

Diagram 2.1 Flowchart

7. Result

7.2.1 TELEGRAM IOT

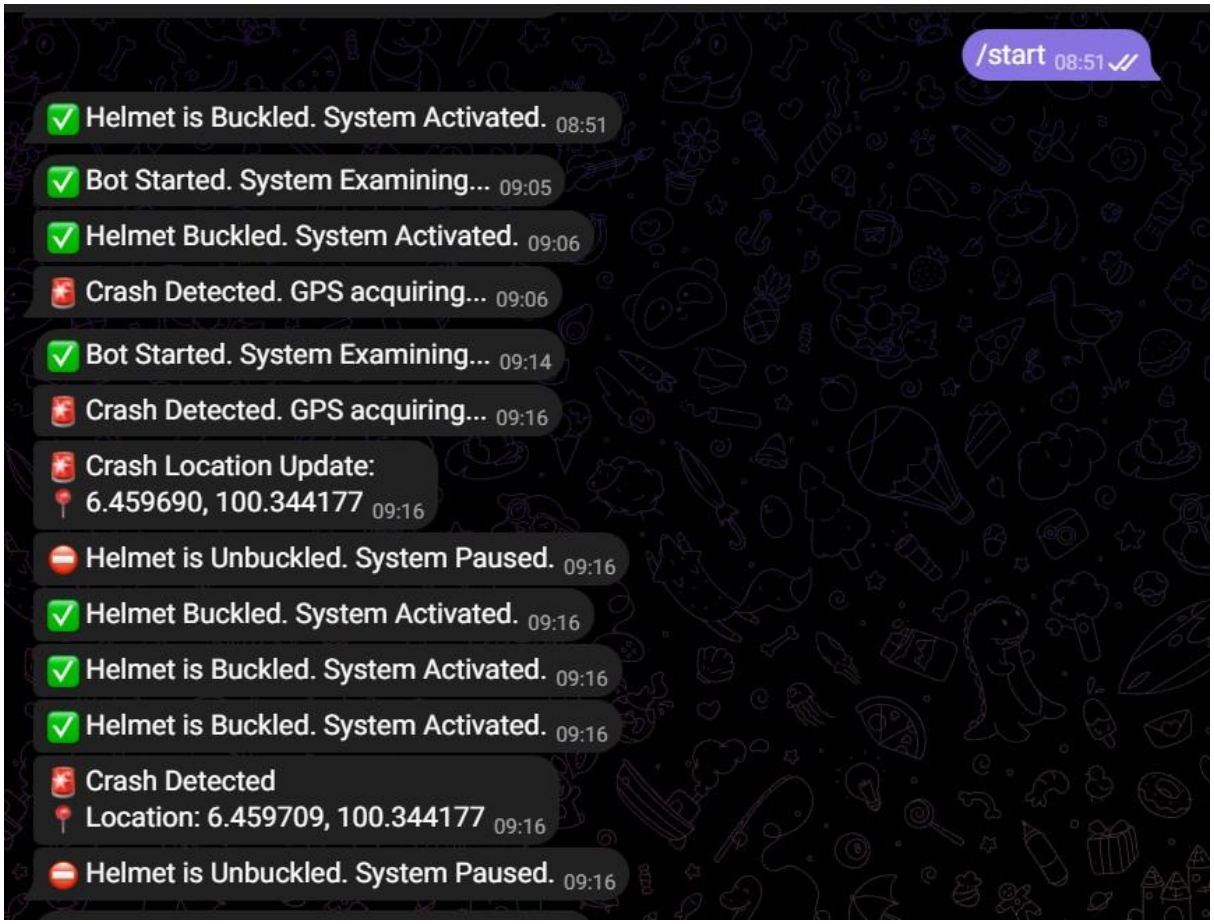

Diagram 3.1 Telegram Interface

The Telegram bot interface provides real-time monitoring and alert capabilities for the RESQ Helmet smart safety system via Raspberry Pi Pico W. It sends immediate notifications when the helmet is buckled or unbuckled, indicating the system's active or paused status. Upon detecting a crash through the MPU6050 sensor, the bot automatically triggers a high-priority alert with the rider’s precise GPS coordinates, enabling rapid location tracking. These automated messages confirm system responsiveness and ensure that emergency notifications are issued without delay. The use of Telegram for this system offers a lightweight and accessible method for real-time safety updates, making it suitable for fast, remote emergency communication

7.2.2 LDR

Condition	LDR Lux Value	Action
Bright Area	> 20.0 lux	Red LED = OFF
Medium Light	10–20 lux	Red LED = OFF
Dark (Tunnel/Night)	< 10.0 lux	Red LED = ON (warning)

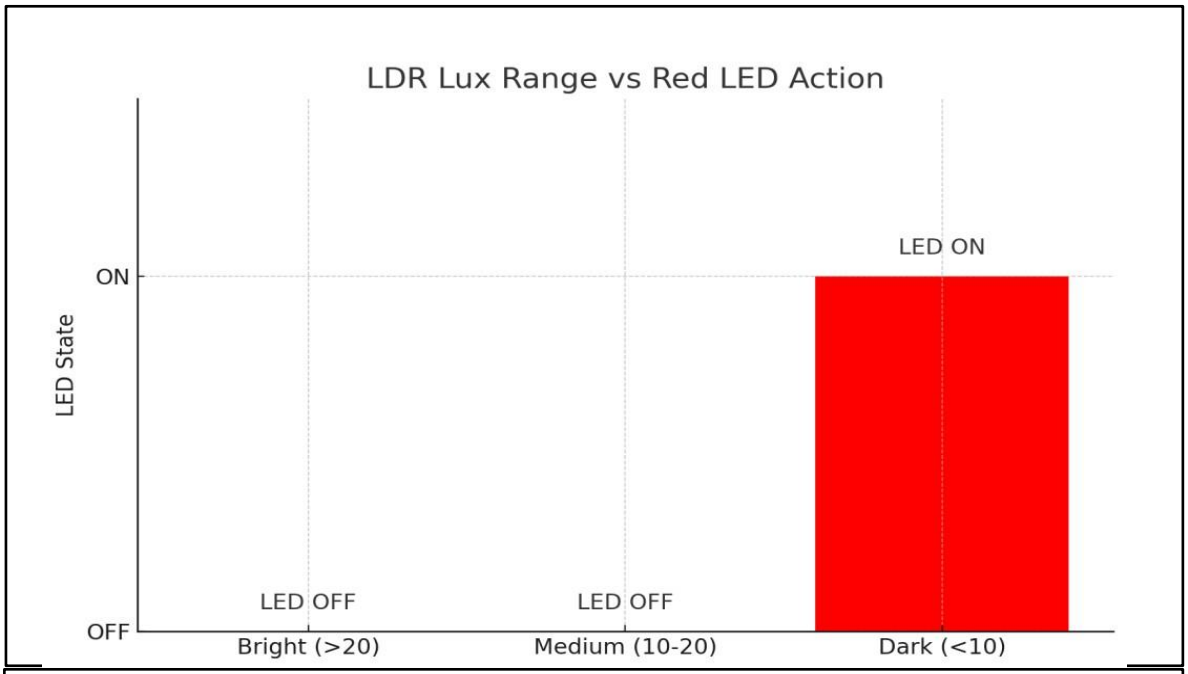

Diagram 4 .1 Graph LDR vs Red LED

7.2.3 GPS

```
Fix!
Location: 6.460302, 100.353546
Altitude: 60.1 meters
Speed: 0.067 knots
UTC Time: struct_time(tm_year=2025, tm_mon=7, tm_mday=24, tm_hour=8, tm_min=33, tm_sec=33, tm_wday=0, tm_yday=0, tm_isdst=-1)
```

Diagram 5.1 GPS

7.2.4 MPU6050

```
Accel: (3.61285, -2.38223, -8.62152) | Gyro: (-0.0917963, 0.0418346, 0.00799387)
```

1. Acceleration Magnitude

$$\mathbf{acc} = (3.61285, -2.38223, -8.62152)$$

$$acc_magnitude = \sqrt{(3.61285)^2 + (-2.38223)^2 + (-8.62152)^2}$$

Break it down:

- $3.61285^2 = 13.055$
- $(-2.38223)^2 = 5.672$
- $(-8.62152)^2 = 74.355$

Sum:

$$13.055 + 5.672 + 74.355 = 93.082$$

Square root:

$$\sqrt{93.082} \approx \boxed{9.643}$$

2. Gyro Magnitude

$$\mathbf{gyro} = (-0.0917963, 0.0418346, 0.00799387)$$

$$gyro_magnitude = \sqrt{(-0.0917963)^2 + (0.0418346)^2 + (0.00799387)^2}$$

Break it down:

- $(-0.0917963)^2 = 0.008428$
- $0.0418346^2 = 0.00175$
- $0.00799387^2 = 0.000064$

Sum:

$$0.008428 + 0.00175 + 0.000064 = 0.010242$$

Square root:

$$\sqrt{0.010242} \approx \boxed{0.1012}$$

Diagram 6.1 MPU6050

7.2.5 PROTOTYPE

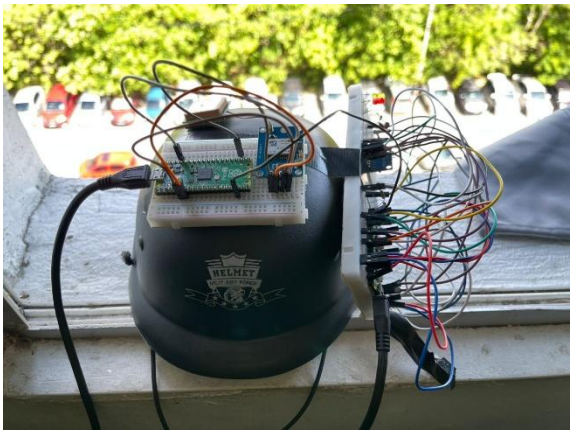

Diagram 7.1

The model consists of a real motorcycle helmet connected to a breadboard-based electronic system controlled by a Raspberry Pi Pico W. The setup integrates an MPU6050 sensor for impact detection, a GPS module for real-time location tracking, a limit switch simulating the helmet buckle, an LED indicator, a buzzer for alerts, and an LDR sensor for ambient light detection. All components are interconnected using jumper wires and powered through a laptop USB connection. The wiring structure reflects modular design principles, allowing the system to activate only when the helmet is worn (limit switch pressed), detect crashes using motion data from the MPU6050, and send emergency alerts that include the rider's GPS coordinates. The buzzer and LED serve as local alerts for nearby assistance. This functional prototype effectively demonstrates how IoT-based safety technology can enhance motorcycle rider protection and enable faster emergency response in real accident scenarios.

8 Discussion

The System functioned use RTOS to ensured task separation without lag. Crash detection and GPS communication were reliable. The LDR component enhanced night-time safety, a feature often overlooked in helmet systems. The switch prevented false triggers when helmet was not worn. Overall, performance met project expectations

9 Conclusion

The ResQ Helmet project successfully integrates crash detection, automated emergency alerting, and night safety lighting into a compact wearable system. It enhances rider safety by reducing emergency response time and improving visibility. The system supports SDG 11 by promoting safer, smarter urban mobility.

10 Recommendations

Moving forward, several improvements and enhancements can be made to the ResQ Helmet system to increase its effectiveness, scalability, and user convenience. One area of development includes the integration of a mobile application that allows emergency contacts to monitor helmet status and receive live location updates. This would extend the reach of the alert system beyond basic messaging.

Another suggestion is to incorporate a rechargeable lithium-ion battery with power management circuitry to improve the system's runtime and support longer journeys. In terms of hardware design, both Raspberry Pi Pico W boards could be integrated into a single custom PCB, reducing wiring complexity and improving reliability.

Furthermore, integrating a cloud-based system could enable data logging and analysis of crash events. This would be beneficial for accident forensics and improving future safety standards. Additional features such as rear-facing LED indicators or automated turn signals could also be added to further enhance visibility and rider awarenes By implementing these enhancements, the ResQ Helmet can evolve from a prototype into a highly functional, market-ready safety device for everyday motorcyclists.

11 References

[1] R. H. Elabd, M. M. Darwish, H. H. Alshamekh, A. S. Moawad, dan M. M. Younis, "A Survey of IoT-Based Smart Helmet Technologies for Motorcycle Rider's Safety," Journal of Engineering Research and Reports, vol. 27, no. 5, hlm. 72–81, Mei 2025. journaljerr.com

[2] S. M. Kayser Mehbub Siam, K. I. Sumaiya, Md R. Al-Amin, T. H. Turjo, A. I. Islam, A. H. M. A. Rahim, dan Md R. Hasan, "Re al-time accident detection and physiological signal monitoring to enhance motorbike safety and emergency response," arXiv preprint, 28 Mar. 2024. arXiv

[3] V. Kumar dan N. B. E. Narasimhayya, "Smart Helmet using GSM & GPS Technology for Accident Detection and Reporting System," Conference Paper, Des. 2022 (prospektif diakses), IEEE atau platform sejenis. IJERT+3ResearchGate+3ResearchGate+3

[4] M. Intal Tayag dan M. E. Asuncion De Vigal Capuno, "Smart Motorcycle Helmet: Real-Time Crash Detection with Emergency Notification, Tracker and Anti-Theft System using IoT Cloud based Technology," International Journal of Computer Science and Information Technology, vol. 11, no. 3, hlm. 81–94, Jun. 2019. IEEE Xplore+7iScholar+7iScholar+7

[5] "Konnect: An Internet of Things(IoT) based smart helmet for accident detection and notification," Conference Paper, S. Chandran, S. Chandrasekar, dan N. Edna Elizabeth, Des. 2016. ResearchGate

https://github.com/mymadi/FKTE-Cytron

12 Appendix

12.2 Thonny coding(PICO W SERVER)

```
import time
import board
import analogio
import digitalio
import pyRTOS
import busio
import wifi
import socketpool
import ssl
import adafruit_requests
from helmet_system import LightMonitor, MPUCrashDetector, BuzzerAlert, TelegramBot, HelmetGPS

# --- WiFi Connect ---
print("📶 Connecting to WiFi...")
wifi.radio.connect("Redmi Note 10", "lhna1234")
pool = socketpool.SocketPool(wifi.radio)
requests = adafruit_requests.Session(pool, ssl.create_default_context())
print("✅ Connected to WiFi:", wifi.radio.ipv4_address)
```

```
limit_switch = digitalio.DigitalInOut(board.GP14)
limit_switch.direction = digitalio.Direction.INPUT
limit_switch.pull = digitalio.Pull.UP  # Active LOW

yellow_led = digitalio.DigitalInOut(board.GP19)
yellow_led.direction = digitalio.Direction.OUTPUT

# ——— Sensors ———
i2c = busio.I2C(scl=board.GP3, sda=board.GP2)

# --- Class Instances ---
light_sensor = LightMonitor(ldr_pin=board.GP26, red_led=board.GP0, dark_thresh=10.0, light_thresh=20.0)
crash_detector = MPUCrashDetector(i2c)
buzzer = BuzzerAlert(board.GP18)

# ——— GPS via UART ———
uart_gps = busio.UART(board.GP4, board.GP5, baudrate=9600, timeout=10)
gps_module = HelmetGPS(uart_gps)


# --- Telegram Setup ---
bot = TelegramBot("Redmi Note 10", "lhna1234", "7389065044:AAGhuABAEoXIslYOzsxQOTZ-6e9tZK2hPZA")
bot.requests = requests
bot.wait_for_chat_id()
bot.send_message("✅ Bot Started. System Examining...")

# --- Helmet State ---
helmet_state = {
    "started": False,
    "crash": False,
    "pending_crash": False,
    "lux": None,
    "gps": None,
}
```

```python
# --- RTOS Tasks ---

def telegram_task(self):
    yield
    while True:
        cmd = telegram_get_updates()
        if cmd == "/start" and not helmet_state["started"] and not limit_switch.value:
            helmet_state["started"] = True
            bot.send_message("✅ Helmet Buckled. System Activated.")
        yield [pyRTOS.timeout(1.0)]

def light_task(self):
    yield
    while True:
        if helmet_state["started"] and not limit_switch.value:
            lux = light_sensor.check_light()
            helmet_state["lux"] = lux
        else:
            helmet_state["lux"] = None
            light_sensor.led.value = False
            light_sensor.is_dark = False  # sync state so LED doesn't trigger wrongly on next cycle
        yield [pyRTOS.timeout(1.0)]
```

```python
# --- Telegram Polling Function ---
def telegram_get_updates():
    try:
        url = f"https://api.telegram.org/bot{bot.token}/getUpdates"
        resp = bot.requests.get(url).json()
        for update in resp.get("result", []):
            msg = update.get("message", {})
            if msg:
                text = msg.get("text")
                if text:
                    return text
    except Exception as e:
        print(f"Error getting updates: {e}")
    return None

# --- RTOS Tasks ---
```

```python
def crash_task(self):
    yield
    crash_active = False
    while True:
        if helmet_state["started"] and not limit_switch.value:
            if not crash_active:
                crash, acc, gyro = crash_detector.check_crash()
                print(f"☑ Accel: {acc} | 🌀 Gyro: {gyro}")
                yield [pyRTOS.timeout(0.1)]

                # Simulated crash for testing
                crash = True

                if crash:
                    crash_active = True
                    helmet_state["crash"] = True
                    yellow_led.value = True
                    buzzer.on()

                    loc = helmet_state.get("gps")
                    if loc:
                        msg = f"🚨 Crash Detected\n📍 Location: {loc['Latitude']:.6f}, {loc['Longitude']:.6f}"
                        print(f"🚨 Crash Detected at {loc['Latitude']:.6f}, {loc['Longitude']:.6f}")
                        bot.send_message(msg)
                    else:
                        bot.send_message("🚨 Crash Detected. GPS acquiring...")
                        print("🚨 Crash Detected. GPS acquiring...")
                        helmet_state["pending_crash"] = True
        else:
            yellow_led.value = False
            buzzer.off()
            crash_active = False

        yield [pyRTOS.timeout(0.1)]
```

```python
# --- RTOS Tasks ---

def telegram_task(self):
    yield
    while True:
        cmd = telegram_get_updates()
        if cmd == "/start" and not helmet_state["started"] and not limit_switch.value:
            helmet_state["started"] = True
            bot.send_message("✅ Helmet Buckled. System Activated.")
        yield [pyRTOS.timeout(1.0)]

def light_task(self):
    yield
    while True:
        if helmet_state["started"] and not limit_switch.value:
            lux = light_sensor.check_light()
            helmet_state["lux"] = lux
        else:
            helmet_state["lux"] = None
            light_sensor.led.value = False
            light_sensor.is_dark = False  # sync state so LED doesn't trigger wrongly on next cycle
        yield [pyRTOS.timeout(1.0)]

def gps_task(self):
    yield
    last_update = time.monotonic()
    while True:
        gps_module.update()
        now = time.monotonic()
        if now - last_update >= 1.0:
            last_update = now
            if gps_module.has_fix():
                loc = gps_module.get_location()
                helmet_state["gps"] = loc
                print(f"📍 GPS Fix: {loc['Latitude']:.6f}, {loc['Longitude']:.6f}")
                if helmet_state.get("pending_crash"):
                    bot.send_message(f"🚨 Crash Location Update:\n📍 {loc['Latitude']:.6f}, {loc['Longitude']:.6f}")
                    helmet_state["pending_crash"] = False
            else:
                print("❌ GPS: No fix")
        yield [pyRTOS.timeout(1.0)]

def limit_switch_task(self):
    yield
    last_state = limit_switch.value  # HIGH = unbuckled, LOW = buckled

    while True:
        current = limit_switch.value
        if current != last_state:
            if not current:  # LOW = Helmet Buckled
                helmet_state["started"] = True
                bot.send_message("✅ Helmet is Buckled. System Activated.")
                print("✅ Helmet worn. System resumed.")
            else:  # HIGH = Helmet Unbuckled
                helmet_state["started"] = False
                helmet_state["crash"] = False
                helmet_state["pending_crash"] = False
                bot.send_message("⛔ Helmet is Unbuckled. System Paused.")
                print("⛔ Helmet removed. System paused.")
            last_state = current

        yield [pyRTOS.timeout(0.5)]
# --- RTOS Task Scheduler ---
pyRTOS.add_task(pyRTOS.Task(telegram_task, name="TelegramTask", priority=1))
pyRTOS.add_task(pyRTOS.Task(light_task, name="LightTask", priority=1))
pyRTOS.add_task(pyRTOS.Task(crash_task, name="CrashTask", priority=1))
pyRTOS.add_task(pyRTOS.Task(limit_switch_task, name="SwitchMonitor", priority=1))
pyRTOS.add_task(pyRTOS.Task(gps_task, name="GPS", priority=2))

pyRTOS.start()
```

Diagram 8.1 Coding